Chapitre : Les Fichiers en C

Par Robert DIASSÉ

Introduction aux Fichiers en C

Les fichiers sont utilisés pour **stocker des données de manière persistante**. En langage C, la bibliothèque `<stdio.h>` fournit des fonctions pour manipuler des fichiers.

Deux types de fichiers principaux :

1. **Fichiers texte** : Lisibles par l'humain, utilisent des encodages comme ASCII.
2. **Fichiers binaires** : Contiennent des données au format brut, non directement lisibles.

---

1. Ouverture et Fermeture de Fichiers

Pour manipuler un fichier, il faut :

1. **Ouvrir le fichier** avec `fopen`.
2. **Effectuer des opérations** (lecture, écriture...).
3. **Fermer le fichier** avec `fclose`.

Modes d'ouverture avec `fopen` :

Mode	Description
`r`	Lecture seule. Erreur si le fichier n'existe pas.
`w`	Écriture seule. Crée un fichier vide ou écrase l'existant.
`a`	Ajout (append). Écrit à la fin sans modifier le contenu existant.
`r+`	Lecture et écriture. Erreur si le fichier n'existe pas.
`w+`	Lecture et écriture. Crée un fichier vide ou écrase l'existant.
`a+`	Lecture et ajout. Lecture possible, écriture à la fin.

Exemple d'ouverture et de fermeture :

```c
#include <stdio.h>

int main() {
    FILE *fichier = fopen("donnees.txt", "r"); // Ouvre en lecture
    if (fichier == NULL) {
        printf("Erreur : impossible d'ouvrir le fichier.\n");
        return 1;
    }

    // Utilisation du fichier...

    fclose(fichier); // Ferme le fichier
    return 0;
}
```

---

2. Lecture de Fichiers

a) Lire caractère par caractère : `fgetc`

Lit un caractère à la fois.

```c
FILE *fichier = fopen("exemple.txt", "r");
int caractere;
while ((caractere = fgetc(fichier)) != EOF) {
    putchar(caractere); // Affiche le caractère
}
```

b) Lire une ligne entière : `fgets`

Lit une ligne jusqu'à un certain nombre de caractères.

```c
FILE *fichier = fopen("exemple.txt", "r");
char ligne[100];
while (fgets(ligne, sizeof(ligne), fichier) != NULL) {
    printf("%s", ligne); // Affiche la ligne
}
```

c) Lire des données formatées : `fscanf`

Lit des données avec un format spécifique.

```c
FILE *fichier = fopen("donnees.txt", "r");
int entier;
float reel;
```

```
char texte[50];
fscanf(fichier, "%d %f %s", &entier, &reel, texte);
printf("Entier : %d, Réel : %.2f, Texte : %s\n", entier, reel, texte);
```

d) Lire des blocs de données : `fread`

Utile pour les fichiers binaires.

```
FILE *fichier = fopen("donnees.bin", "rb");
char buffer[1024];
size_t tailleLue = fread(buffer, 1, sizeof(buffer), fichier);
```

---

3. Écriture dans les Fichiers

a) Écrire caractère par caractère : `fputc`

Écrit un caractère dans un fichier.

```
FILE *fichier = fopen("nouveau.txt", "w");
for (char c = 'A'; c <= 'Z'; c++) {
    fputc(c, fichier); // Écrit A à Z
}
```

b) Écrire une chaîne : `fputs`

Écrit une chaîne entière.

```
FILE *fichier = fopen("nouveau.txt", "w");
fputs("Bonjour, fichier !", fichier);
```

c) Écrire des données formatées : `fprintf`

Écrit des données formatées.

```
FILE *fichier = fopen("donnees.txt", "w");
int entier = 42;
float reel = 3.14;
fprintf(fichier, "Entier : %d, Réel : %.2f\n", entier, reel);
```

d) Écrire des blocs de données : `fwrite`

Utilisé pour les fichiers binaires.

```
FILE *fichier = fopen("donnees.bin", "wb");
char buffer[] = "Données binaires.";
fwrite(buffer, sizeof(char), sizeof(buffer), fichier);
```

---

4. Fonctions Utiles pour la Gestion des Fichiers

a) Vérifier la fin d'un fichier : `feof`

```
if (feof(fichier)) {
    printf("Fin du fichier atteinte.\n");
}
```

b) Vider un tampon : `fflush`

Force l'écriture immédiate des données.

```
fflush(fichier);
```

c) Obtenir la position actuelle : `ftell`

Renvoie la position actuelle du curseur en octets.

```
long position = ftell(fichier);
printf("Position : %ld\n", position);
```

d) Revenir au début : `rewind`

Réinitialise le curseur au début du fichier.

```
rewind(fichier);
```

e) Déplacer le curseur : `fseek`

Déplace le curseur à une position spécifique.

```
fseek(fichier, 20, SEEK_SET); // 20 octets depuis le début
```

---

5. Exemples Pratiques

Exemple 1 : Copier le contenu d'un fichier

```
void copierFichier(const char *source, const char *destination) {
    FILE *fsource = fopen(source, "r");
    FILE *fdestination = fopen(destination, "w");
    if (fsource == NULL || fdestination == NULL) {
        printf("Erreur d'ouverture de fichier.\n");
        return;
    }

    char caractere;
    while ((caractere = fgetc(fsource)) != EOF) {
        fputc(caractere, fdestination);
    }

    fclose(fsource);
    fclose(fdestination);
}
```

Exemple 2 : Compter le nombre de lignes dans un fichier

```
int compterLignes(const char *nomFichier) {
    FILE *fichier = fopen(nomFichier, "r");
    if (fichier == NULL) {
        printf("Erreur d'ouverture du fichier.\n");
        return -1;
    }

    int lignes = 0;
    char ligne[256];
    while (fgets(ligne, sizeof(ligne), fichier) != NULL) {
        lignes++;
    }

    fclose(fichier);
    return lignes;
}
```

Conclusion

Ce cours vous donne toutes les bases nécessaires pour :

1. Lire, écrire et modifier des fichiers texte et binaires.

2. Manipuler les fichiers en utilisant les fonctions standards.
3. Traiter des données formatées ou brutes efficacement.

Entraînez-vous avec les exemples pratiques pour maîtriser les bases de la gestion des fichiers en C !